TOUR PLANNER Protocol

Software Engineering 2 Labor

Zeynep Bera Tokel,

Berceste Yagmur Aslan Özugur

Git Link: https://github.com/zeybera/tourplanner-frontend

1. Purpose

This protocol explains the frontend structure of the Tour Planner application. This application is built with Angular and uses a reactive approach with signals.

2. Design Pattern for Frontend

This application follows the **Model-View-ViewModel (MVVM)** pattern.

Model

In this project, the Model consists of both the data definition and the state of the application. The data structure is defined using TypeScript through the Tour interface, which specifies all properties of a tour.

The state is managed using signals inside the TourService. The list of tours and the selected tour ID are stored centrally, making the service the single source of truth. As a result, all components work with the same data rather than maintaining their own local state.

All changes to the state stored in signals are made through methods such as create, update, and delete in the service. States are shared as readonly signals, so components can read the data but cannot change it directly. This helps keep the data consistent and avoids mistakes.

ViewModel

In this project, the ViewModel connects the View with the Model and controls how data flows between them.

It handles user interactions through event handlers in the components. For example, when a user types into an input field, the event handler takes the entered value and updates a signal that stores this value. When a button is clicked, the event handler triggers an action such as creating or updating a tour by calling a method in the service.

Event handlers call service methods such as create, update, and delete, and the service updates the state using signals. Components do not change the data directly and instead use the service for all updates.

Validation is handled using computed signals. These signals check whether all required fields are filled correctly and automatically enable or disable actions like saving a tour.

View

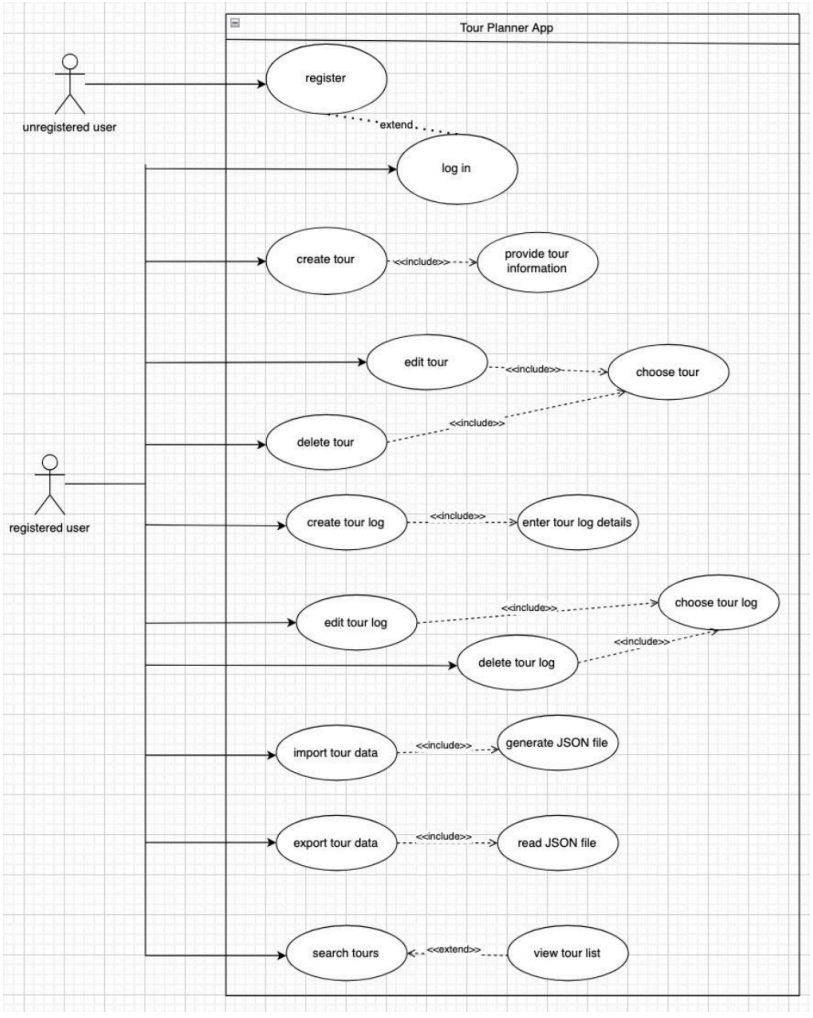
In this project, view is responsible for displaying data and capturing user input.

It receives data from the component, which gets signals from the TourService and makes them available in the template.In the template, these signals are called (e.g., tours() or selectedTour()) to retrşieve the current values and display them in the UI.

User interactions, such as typing into input fields or clicking buttons, are captured in the View and passed to the ViewModel through event bindings. The View itself does not contain business logic or directly modify the data.

View reacts automatically to changes in the signals. When the state changes in the service, the updated values are reflected in the template without requiring manual updates or page reloads.

3. Use Cases



The use case diagram above illustrates the planned functionalities of the Tour Planner application. At the current stage (intermediate hand-in), not all features shown in the diagram are implemented yet. User roles such as registered and unregistered users are not yet fully integrated into the system.

So far, the core features that have been implemented include creating, editing, and deleting tours, as well as basic tour selection and display. The user can create a tour by entering the required tour data such as description, start location, destination, transport type, and route information.

Once a tour is created, it can be selected from the list and viewed in detail. The user can then update the tour information or delete the tour. To perform these actions, the user must first select a specific tour.

In addition, tour logs have been implemented. Users can create logs for a selected tour by entering information such as date, comment, difficulty, and rating. Each log is linked to a specific tour using a tourId. Tour logs can also be viewed, edited, and deleted. To perform editing or deletion, the user must first select a specific tour log.

Other functionalities shown in the use case diagram, such as search and import/export features, are planned but not yet implemented and will be completed in the final submission.

4. User Experience

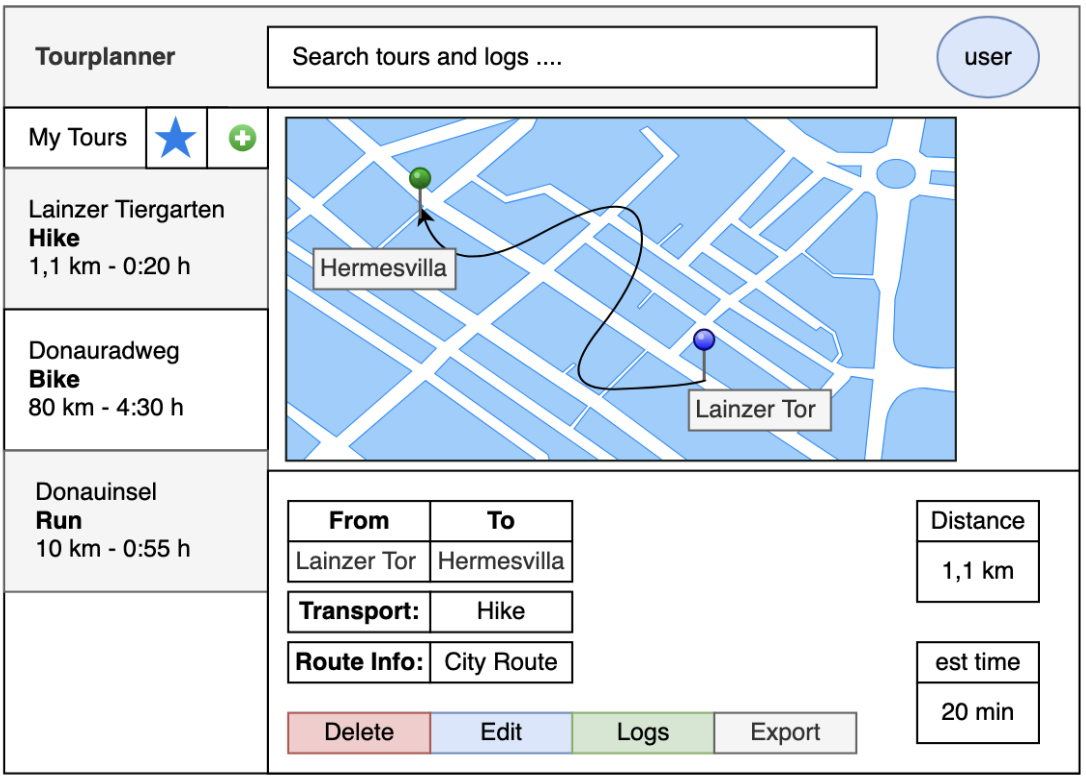


Figure 1

The wireframe above illustrates the main page of the Tour Planner application and shows how the user interacts with tours, their details and tour logs.

The layout is divided into two main areas:

- On the left side, the user can see a list of tours. This list provides a quick overview of the available tours and allows the user to select one of them.

- On the right side, the details of the selected tour is displayed. These details include route-related information, transport type, distance, estimated time, and available actions such as viewing, deleting, exporting, or adding a tour log.

Central part of the planned interface is the map area, which is intended to visualize the route of the selected tour. At the current stage, however, the map integration has not yet been completed.

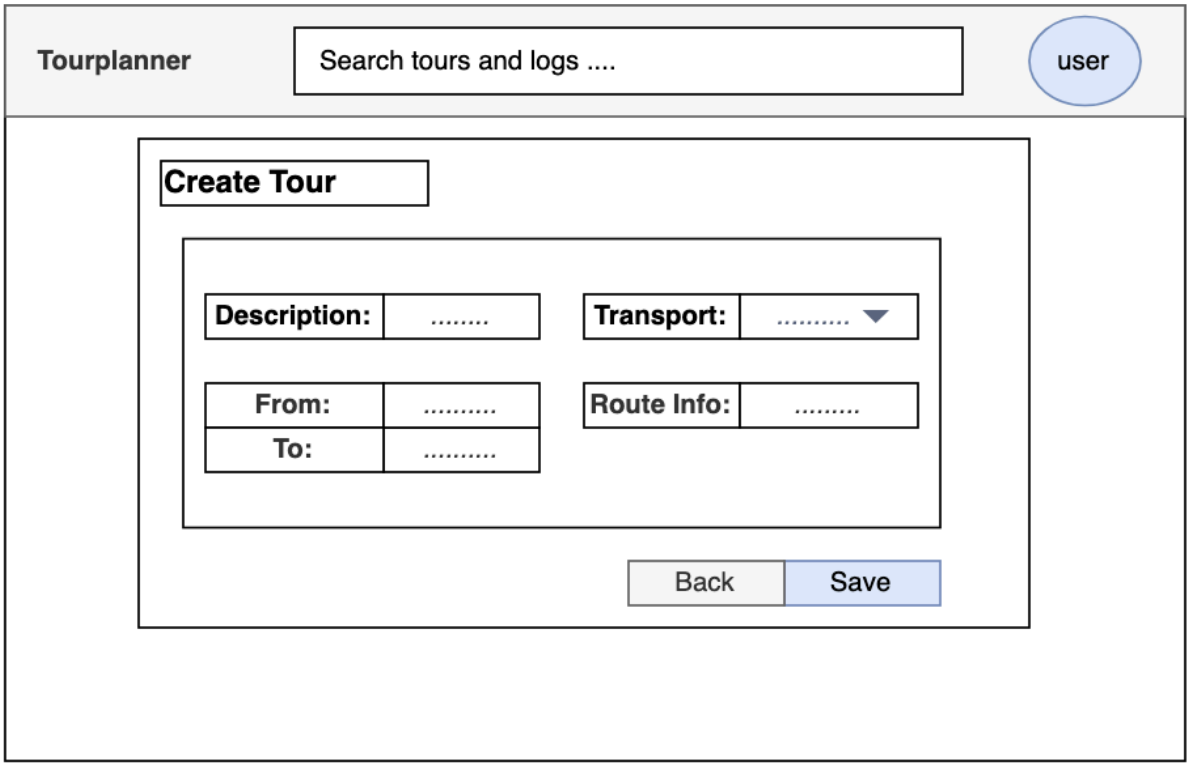


Figure 2

This wireframe above illustrates the create tour view, which is opened after the user clicks the plus button on the main page (on Figure 1). This view allows the user to enter the required information for a new tour. At the bottom of the form, the user can either go back to the previous page or save the new tour.

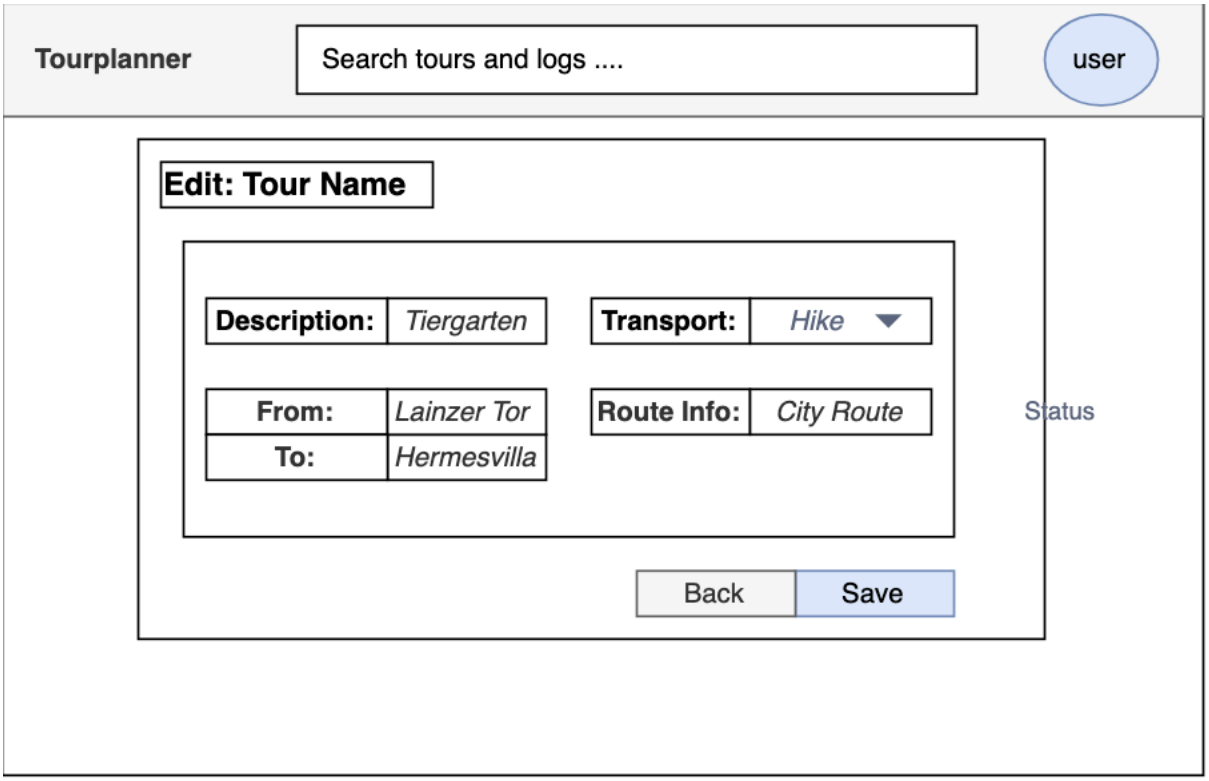
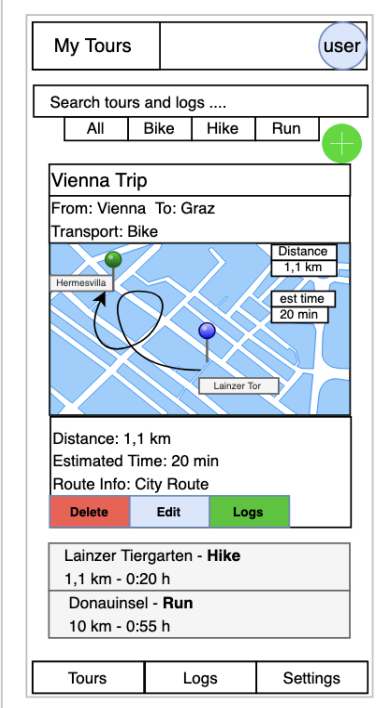


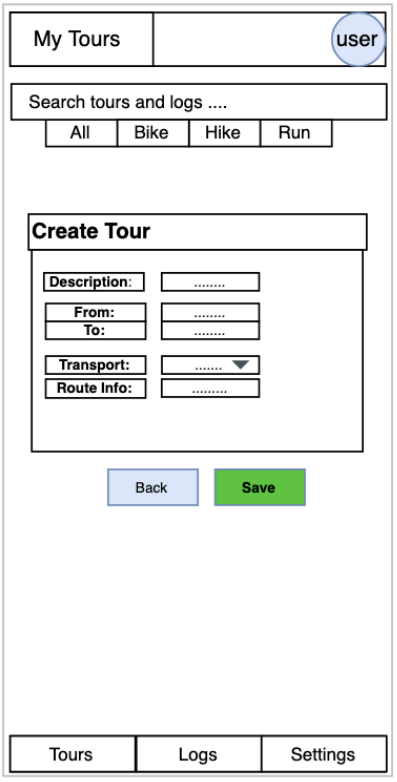
Figure 3

This wireframe above illustrates the edit tour view, which is opened when the user selects a tour from the tour list and then clicks the edit button on the main page (Figure 1). In this view, the user can modify the existing tour information.



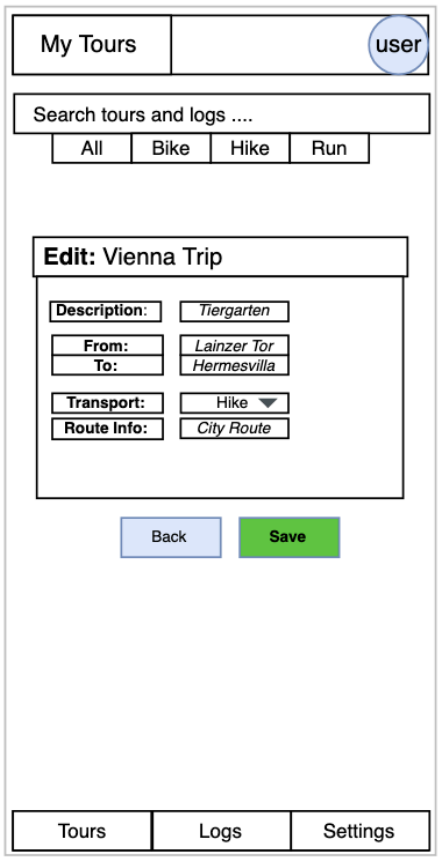
This wireframe on the left shows the mobile version of the main page. As part of a responsive design, all content is arranged from top to bottom so the user can easily use the application on a mobile device.

Figure 4



This wireframe on the left shows the mobile version of the create tour view. It provides the same functionality as the desktop version and follows a responsive design for mobile devices.

Figure 5



This wireframe on the left shows the mobile version of the edit tour view. It provides the same functionality as the desktop version and follows a responsive design for mobile devices.

Figure 6